

Hackathon Management System

A Database Management Systems (DBMS) Mini Project

Submitted by: **[Your Name]**

Roll No: **[Roll Number]**

Class: **[Class / Division]**

Subject: **Database Management Systems**

Faculty: **Dr. Mrudula Gudadhe**

Academic Year: **2026-27**

GitHub: **[paste repository link]**

1. Introduction

A hackathon involves many participants forming teams, submitting projects and getting scored. Managing this on paper or spreadsheets causes duplicate entries, oversized teams and calculation mistakes. This project stores the data in a MySQL relational database and provides a simple Streamlit web interface for Create, Read, Update and Delete (CRUD) operations.

2. Problem Statement

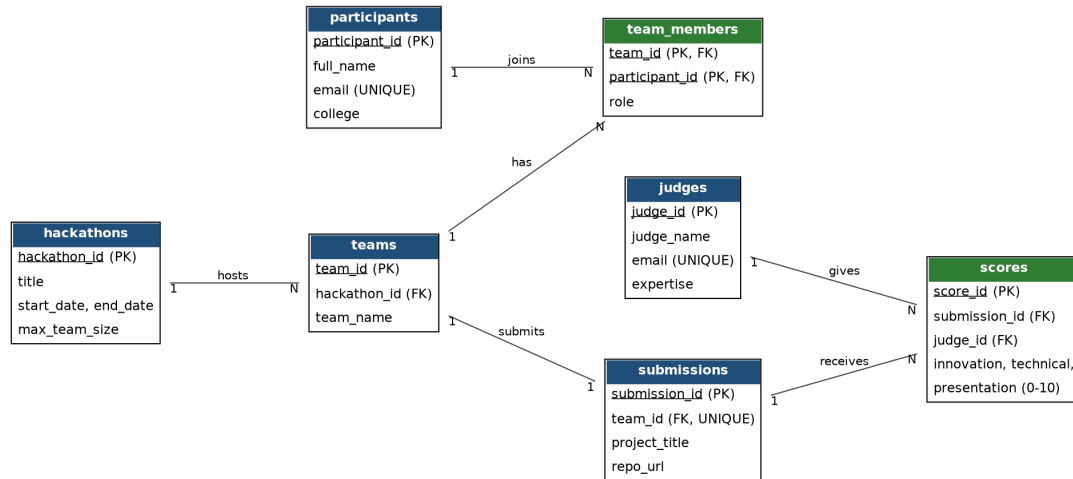
To design and implement a normalized relational database and an application to manage hackathons, participants, teams, project submissions and judge scores, with rules (such as maximum team size and one team per participant) enforced inside the database and an automatically ranked leaderboard.

3. Objectives

- Design a relational schema with proper keys, constraints and normalization up to 3NF.
- Provide a user-friendly interface for CRUD operations.
- Use triggers to enforce the maximum team size and one team per participant in a hackathon.
- Let multiple judges score each project and rank teams by their average score.
- Use a stored procedure with a transaction to register a team and its leader together.
- Use views to show team sizes and a ranked leaderboard (RANK window function).
- Demonstrate joins, GROUP BY and subqueries through reports.

4. ER Diagram

The system has 7 entities. **team_members** resolves the many-to-many relationship between participants and teams; **scores** resolves the many-to-many relationship between judges and submissions. Each team can have at most one submission.



- Hackathon 1 : N Teams
- Team 1 : N Team_Members, Participant 1 : N Team_Members (M:N between teams and participants)
- Team 1 : 1 Submission
- Submission 1 : N Scores, Judge 1 : N Scores (M:N between judges and submissions)

5. Database Schema

Table	Columns	Constraints
participants	participant_id, full_name, email, college	PK participant_id; UNIQUE email
hackathons	hackathon_id, title, start_date, end_date, max_team_size	PK hackathon_id; CHECK end_date >= start_date; CHECK max_team_size 1-6
teams	team_id, hackathon_id, team_name	PK team_id; FK hackathon_id; UNIQUE(hackathon_id, team_name)
team_members	team_id, participant_id, role	Composite PK(team_id, participant_id); FKs to teams, participants
submissions	submission_id, team_id, project_title, repo_url	PK submission_id; FK team_id (UNIQUE: one per team)
judges	judge_id, judge_name, email, expertise	PK judge_id; UNIQUE email
scores	score_id, submission_id, judge_id, innovation, technical, presentation	PK score_id; FKs; UNIQUE(submission_id, judge_id); CHECK each score 0-10

Normalization

- **1NF:** all columns are atomic; team members are stored as rows, not lists.
- **2NF:** in team_members (composite key) the non-key attribute role depends on the whole key.
- **3NF:** no transitive dependencies; hackathon details are stored once and referenced by foreign key; the total score is calculated in a view instead of being stored.

6. Technologies Used

Component	Technology
Database	MySQL 8
Backend / logic	Python 3, mysql-connector-python, pandas
User interface	Streamlit
Version control	Git and GitHub

7. SQL Queries and Features Implemented

7.1 Triggers

trg_team_full – blocks a new member when the team already has max_team_size members:

```
CREATE TRIGGER trg_team_full BEFORE INSERT ON team_members
FOR EACH ROW BEGIN
    DECLARE v_max INT; DECLARE v_cnt INT;
    SELECT h.max_team_size INTO v_max
    FROM teams t JOIN hackathons h ON h.hackathon_id = t.hackathon_id
    WHERE t.team_id = NEW.team_id;
    SELECT COUNT(*) INTO v_cnt FROM team_members WHERE team_id = NEW.team_id;
    IF v_cnt >= v_max THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Team is full';
    END IF;
END
```

trg_one_team_per_hackathon – blocks a participant from joining a second team in the same hackathon:

```
CREATE TRIGGER trg_one_team_per_hackathon BEFORE INSERT ON team_members
FOR EACH ROW BEGIN
    DECLARE v_dup INT;
    SELECT COUNT(*) INTO v_dup FROM team_members tm
    JOIN teams t1 ON t1.team_id = tm.team_id
    JOIN teams t2 ON t2.team_id = NEW.team_id AND t2.hackathon_id = t1.hackathon_id
    WHERE tm.participant_id = NEW.participant_id;
    IF v_dup > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Participant already in a team for this hackathon';
    END IF;
END
```

7.2 Stored procedure with transaction

```
CREATE PROCEDURE register_team(IN p_hackathon INT, IN p_team_name VARCHAR(80), IN p_leader INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    INSERT INTO teams (hackathon_id, team_name) VALUES (p_hackathon, p_team_name);
    INSERT INTO team_members (team_id, participant_id, role)
    VALUES (LAST_INSERT_ID(), p_leader, 'leader');
    COMMIT;
END
```

The team and its leader are saved together; if the second insert fails, the first is rolled back.

7.3 View: leaderboard

```
CREATE VIEW v_leaderboard AS
SELECT t.hackathon_id, t.team_name, s.project_title,
    COUNT(sc.score_id) AS judges_count,
    ROUND(AVG(sc.innovation + sc.technical + sc.presentation), 2) AS avg_total,
    RANK() OVER (PARTITION BY t.hackathon_id
    ORDER BY AVG(sc.innovation + sc.technical + sc.presentation) DESC) AS team_rank
FROM submissions s
```

```

JOIN teams t ON t.team_id = s.team_id
JOIN scores sc ON sc.submission_id = s.submission_id
GROUP BY t.hackathon_id, t.team_name, s.project_title;

```

7.4 Reporting queries

```

-- JOIN
SELECT h.title, t.team_name, p.full_name, tm.role
FROM team_members tm
JOIN teams t ON t.team_id = tm.team_id
JOIN hackathons h ON h.hackathon_id = t.hackathon_id
JOIN participants p ON p.participant_id = tm.participant_id;

-- GROUP BY
SELECT college, COUNT(*) AS participants FROM participants GROUP BY college;

-- LEFT JOIN: teams that have not submitted
SELECT t.team_name FROM teams t
LEFT JOIN submissions s ON s.team_id = t.team_id WHERE s.submission_id IS NULL;

-- Subquery: teams scoring above average
SELECT team_name, avg_total FROM v_leaderboard
WHERE avg_total > (SELECT AVG(avg_total) FROM v_leaderboard);

-- Judge strictness
SELECT j.judge_name,
       ROUND(AVG(sc.innovation + sc.technical + sc.presentation), 2) AS avg_given
FROM judges j JOIN scores sc ON sc.judge_id = j.judge_id
GROUP BY j.judge_name ORDER BY avg_given;

```

8. Screenshots

Insert screenshots of the running application here:

Fig 8.1 Dashboard

Fig 8.2 Participants - CRUD

Fig 8.3 Team registration

Fig 8.4 Trigger error: Team is full

Fig 8.5 Judges and scoring

Fig 8.6 Leaderboard

Fig 8.7 SQL Reports

9. Conclusion

The project demonstrates the core DBMS concepts of the course: relational design, normalization, constraints, SQL queries, views, a trigger, a stored procedure and transactions. Business rules are enforced by the database itself, keeping data consistent.

10. Future Scope (Optional)

- User login with roles (admin, judge, participant)
- Email notifications for deadlines and results
- Deploying the app on a cloud service